

Algorithm Analysis

→ Intro to Algorithm Analysis

→ Algorithm Analysis is the process of measuring the efficiency of an algorithm in terms of (time) and memory (space) required to solve a problem.

→ Instead of checking how many seconds a program takes on one computer, we analyze how its performance changes as the input size increases.

Why it's important:

- Helps compare different algorithms.
- Choose the most efficient solution.
- Saves execution time.
- Reduce memory usage.
- Essential for solving large-scale problems.

2. Time Complexity!

→ Time complexity is the amount of time an algorithm takes to execute as the size of input (n) increases.

→ It does not measure actual time in seconds.

→ Instead, it measures how the number of operations grows with input size.

Common Time Complexities!

$O(1)$ → Constant Time

$O(\log n)$ → Logarithmic Time

$O(n)$ → Linear Time

$O(n \log n)$ → Linearithmic Time

$O(n^2)$ → Quadratic Time

$O(2^n)$ → Exponential Time

$O(n!)$ → Factorial Time

DELTA
Date / /
Pg No.
e.g. `for (int i = 0; i < n; i++) {`

Runs in (n) time.

Time Complexity = $O(n)$

3. Space Complexity:

Space Complexity is the amount of memory used by an algorithm during execution.

It includes:

- Variable
- Arrays
- Function calls
- Recursion Stack
- Extra Data Structure

Formula:

Space Complexity = Input Space + Auxiliary Space

e.g. `int a, b, c;`

Use constant memory.

Space Complexity = $O(1)$

4. Big O Notation (O)

→ Big O represents the Upper Bound or Worst Case Time Complexity.

It tells the maximum time an algorithm may take

e.g. Linear Search

Worst Case = element is at last position

Time Complexity = $O(n)$

(b). Omega Notation (Ω)

→ Omega represents the Lower Bound or Best Case Time Complexity.

It tells the min time an algo may take.

e.g. Linear Search.

Element found at first position.

Time Complexity = $\Omega(1)$

(c) Theta Notation (Θ)..

- Theta represents the Average Case or Tight Bound.
- It describes the exact growth rate of an algorithm.

e.g. Many algo perform around $\Theta(n)$ on average.

5. Best, Average and Worst Case

→ Best Case

Minim number of operations

e.g.

Linear search finds element at first index.

Complexity = $O(1)$

→ Avg Case:

Expected number of operation for random input.

e.g. Element found somewhere in middle.
Complexity = $O(n)$.

→ Worst Case:-

Max number of operations.

e.g. → Element at last index or not present

Complexity = $O(n)$

6. Rule to Calculate Complexity

1. Ignore Constants.

$$O(2n) \rightarrow O(n)$$

2. Keep Highest Power

$$O(n^2 + n + 5) \rightarrow O(n^2)$$

3. Sequential Statements

$$O(n) + O(n) \rightarrow O(2n) \rightarrow O(n)$$

4. Nested Loops Multiply

$$\text{Outer Loop} = O(n)$$

$$\text{Inner Loop} = O(n)$$

$$= O(n^2)$$

5. Different Variable

```
for (i = 0; i < n; i++)  
for (j = 0; j < m; j++)
```

Complexity.

$O(n+m)$

7. Time Complexity of Loops!

eg-1: `for (int i = 0; i < n; i++)`

Runs n times.

$\hookrightarrow O(n)$. {Ignoring Const}

eg-2: `for (int i = 0; i < n; i += 2)`

Runs $n/2$ times.

$\hookrightarrow O(n)$. {Ignoring Const}

eg-3: `for (int i = n; i > 0; i--)`

Runs n times.

$\hookrightarrow O(n)$.

8. Time Complexity of Nested Loops,

```
for (int i=0; i<n; i++) → n  
{  
    for (int j=0; j<n; j++) → n  
    {  
        }  
    }  
}
```

$$= n \times n \rightarrow (n^2)$$

$$= \text{Time Comp} \rightarrow O(n^2)$$

9. Time Complexity of Logarithmic Loops,

```
for (int i=1; i<n; i*=2)
```

→ Values → 1

2

4

8

16

32

→ Every iteration double

Number of iteration!

$\rightarrow (\log_2 n)$

$\rightarrow$ Time Complexity

$O(\log n)$

10. Space Complexity Basics!

Space Complexity measuring
memory usage

$\rightarrow$ Constant Space!

int a, b, c;

$\rightarrow$ Use fixed memory!

$O(1)$

$\rightarrow$ Linear space

int arr[] = new int [n];

$O(n)$. { Memory depends on n }

SECTION A

Recursive Function!

Every recursive call use stack memory.

If recursion depth is n ,

$O(n)$.

Section-1-

1. Square Pattern!

Time Complexity.
- $O(n^2)$.

Space Complexity.
- $O(1)$.

Explanation - The program uses two nested loops. The outer loop prints each row, while the inner loop prints (n) stars in every row. Since both loops run (n) times, the total number of operations becomes $n \times n$, resulting in $O(n^2)$ time complexity. Only a few variables are used, so it's $O(1)$.

2. Hollow Square Pattern

Time Complexity.

$\sim O(n^2)$

Space Complexity.

$\sim O(1)$

Explanation - The program checks every position of the square using nested loops. For each position, it decides whether to print a star or a space based on boundary conditions. Since all $(n \times n)$ positions are visited, the ~~the~~ time complexity is $O(n^2)$. Only constant extra memory is used.

3. Triangle Pattern

Time Complexity.

$\sim O(n^2)$

Space Complexity.

$\sim O(1)$

Explanation- The outer loop controls the number of rows, and the inner loop prints stars according to the current row number. The total number of printed stars is proportional to $1+2+3+\dots+n$, which is $n(n+1)/2$.

$\therefore$ The overall complexity is $O(n^2)$.
only constant space required

4. Inverted Triangle Pattern

Time Complexity
 $= O(n^2)$

Space Complexity
 $= O(1)$

Explanation- The outer loop starts from (n) and decrease to 1. The inner loop prints stars equal to the current row number. The total number of printed stars is again $n(n+1)/2$, giving a time complexity of $O(n^2)$. The algorithm uses only a few variables, so the space complexity is $O(1)$.

5. Pyramid Pattern

- Time Complexity
 $O(n^2)$
- Space Complexity
 $O(1)$

Exp -> The program uses nested loops to print both space and stars. The outer loop controls the number of rows, while the inner loops print the required space followed by stars. Since the total number of printed characters increase with the no. of rows, overall time comp. is $O(n^2)$.

6. Diamond Pattern

Time Complexity
- $O(n^2)$

Space Complexity
- $O(1)$

Exp - The diamond pattern is created by printing an upper pyramid followed by an inverted pyramid. Both parts use nested loops to print spaces and stars.

Date: / /
Pg. No.

The total number of operations grows proportionally to the square of number of rows, resulting in $O(n^2)$.

7: Half Diamond Pattern.

Time Complexity
- $O(n^2)$

Space Complexity
- $O(1)$

Exp. The pattern is formed by combining an increasing triangle and a decreasing triangle. Nested loops are used in both parts to print the required number of stars in each row. Since the total number of printed characters grows quadratically with the input size, the time complexity is $O(n^2)$. The program uses only constant extra space.

Number Pattern:

Time Complexity
- $O(n^2)$

Space Complexity
- $O(1)$

Exp: The program uses two nested loops. The outer loop prints each row, while the inner loop prints the same number repeatedly in every column. Since both loops execute (n) times, the total no. of operations is $O(n^2)$. Only constant extra memory is used.

2. Incrementing Number Pattern

Time Complexity
- $O(n^2)$

Space Complexity
- $O(1)$

Exp: The outer loop controls the row number and the inner loop prints numbers from (1) to (n) in

every row. As both loops execute (n) times, the total number of operations becomes $O(n^2)$. This algorithm uses only constant extra space.

3. Right Triangle Number Pattern.

Time Complexity,
- $O(n^2)$

Space Complexity,
- $O(1)$

Exp-2 The outer loop controls the number of rows, while the inner loop prints no. from 1 up to the current row number. The total number of printed values is $n(n+1)/2$, which results in $O(n^2)$ time complexity. Only constant extra memory is required.

4. Inverted Diamond Number Pattern

Time Complexity,
- $O(n^2)$

Space Complexity,
 $O(1)$

Ex-2) The outer loop increases the number of printed values in each row, while the inner loop prints the same number repeated according to the current row. Since nested loops are used and the total number of printed values grows proportionally to n^2 , the overall complexity is $O(n^2)$.

3. Sandwich Number Pattern.

Time Complexity
- $O(n^2)$

Space Complexity
- $O(1)$

Ex-3) The pattern is formed by first decreasing and then increasing the number of printed values. Nested loops are used to print the required number in each row. The total number of printed element grows proportionally to the square of input size, resulting in $O(n^2)$ time complexity. Only constant extra memory is used.

6. Incrementing Diamond Number Pattern.

Time Complexity
= $O(n^2)$

Space Complexity
= $O(1)$

Exp-> The pattern is created by combining an increasing and a decreasing number pattern. Nested loops are used to print the required number of values in both halves of diamond. Since the total number of printed elements grows quadratically with the number of rows, the overall time complexity is $O(n^2)$. The algorithm uses only constant extra space.

Week-2 Module

2.

1. Linear Search

Time Complexity

- Best Case: $O(1)$
- Avg Case: $O(n)$
- Worst Case: $O(n)$

Space Complexity

$O(1)$

Exp → Linear search checks the elements one by one from the beginning of the array. If the element is found at the first position, the search completes immediately giving $O(1)$ time. In the worst case, it may have to examine every element, resulting in $O(n)$ time complexity. Since only a few variables are used, the space complexity is $O(1)$.

Better Approach

If the array is sorted, Binary Search can be used because it

reduces the search time to $O(\log n)$.

2. Binary Search

Time Complexity,

- Best Case: $O(1)$
- Avg Case: $O(\log n)$
- Worst Case: $O(\log n)$

Space Complexity,

$O(1)$ (for the iterative version)

Exp → Binary Search repeatedly divides the sorted array into two halves and searches only the relevant half. This reduces the number of comparisons significantly. Since the search space becomes half in every iteration, the time complexity is $O(\log n)$. The iterative complexity uses constant extra memory.

Better Approach

Binary Search is already an efficient searching algo. No significantly better comparison-based search exists for general sorted arrays.

3. Bubble Sort

- Time Complexity

- Best Case: $O(n)$ (with swap optimization)
- Avg Case: $O(n^2)$
- Worst Case: $O(n^2)$

- Space Complexity
 $O(1)$

Exp: Bubble sort repeatedly compares adjacent and swaps them if they are in the wrong order. In the avg and worst cases, multiple passes over the array are required, resulting in $O(n^2)$ time complexity. Since sorting is performed in-place, the extra space required is $O(1)$.

Merge Sort:

Merge Sort, Heap Sort, or Quick Sort are preferred for large datasets because they provide better average performance.

4. Insertion Sort

Time Complexity,

- Best Case: $O(n)$
- Avg Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity,
 $O(1)$

Exp - Insertion Sort builds the sorted portion of the array one element at a time by inserting each element into its correct position. If the array is already sorted, only one comparison per element is needed, resulting in $O(n)$ time. In the worst case, many shifts are required, giving $O(n^2)$ complexity. The algorithm uses constant extra memory.

-> Better Approach

For large unsorted datasets, Merge Sort or Heap Sort provide better performance. However, Insertion Sort is efficient for small or nearly sorted arrays.

Σ Selection Sort.

Time Complexity

- Best Case : $O(n^2)$
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$

Space Complexity $O(1)$

Exp → Selection Sort repeatedly finds the minimum element from the unsorted portion and places it at the correct position. It performs the same number of comparisons regardless of the initial order of the elements, so the best, average, and worst case are all $O(n^2)$. The algorithm sorts the array in-place using constant extra space.

Better Approach

Merge Sort, Quick Sort, or Heap Sort are more efficient for larger datasets because they achieve $O(n \log n)$ avg or worst-case performance.

Week 3 Modules:

1. Merge Sort

Time Complexity

- Best Case : $O(n \log n)$
- Avg Case : $O(n \log n)$
- Worst Case : $O(n \log n)$

• Space Complexity

$O(n)$.

Ex-1 Merge Sort follows the Divide and Conquer approach. It repeatedly divides the array into smaller halves until each subarray contains one element, then merges them in sorted order. The array is divided into $\log(n)$ levels, and each level processes (n) elements during merging. Therefore, the overall Time Complexity is $O(n \log n)$. An additional temporary array is required during merging, giving a Space complexity of $O(n)$.

→ Better Approach

Merge Sort is already an efficient algo, with guaranteed $O(n \log n)$

For in-place sorting where extra memory is limited, Heap Sort can be preferred because it requires only $O(1)$ space.

2. Quick Sort

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Space Complexity

$O(\log n)$ (Due to recursion stack in avg case)

Expt → Quick Sort selects a pivot element and partitions the array into two parts. The same process is applied recursively to both partitions. When the pivot divides the array evenly, the complexity becomes $O(n \log n)$.

Better Approach

→ Using a better pivot selection strategy (such as a random pivot or median-of-three) helps avoid the worst case in most situations.

If guaranteed worst-case performance is required, Merge Sort or Heap Sort is a better choice.

3. Heap Sort

- Time Complexity

- Best Case : $O(n \log n)$
- Avg Case : $O(n \log n)$
- Worst Case : $O(n \log n)$

- Space Complexity
 $O(1)$

Exp → Heap Sort first builds a Max Heap from the input array and then repeatedly swaps the root element with the last element and re-heapifies the array.

DELTA
Date / /
Pg No.

After every swap, Heapify restores the heap property. Building the heap takes $O(n)$ time while each Heapify operation takes $O(\log n)$ and is performed for all elements.

$\therefore$ The total time complexity is $O(n \log n)$.

$\rightarrow$ Better Approach:

Heap Sort is already efficient and guarantees $O(n \log n)$ Time Complexity.

In practice, Quick Sort is often faster because of better cache performance.

Section – B

Question 1

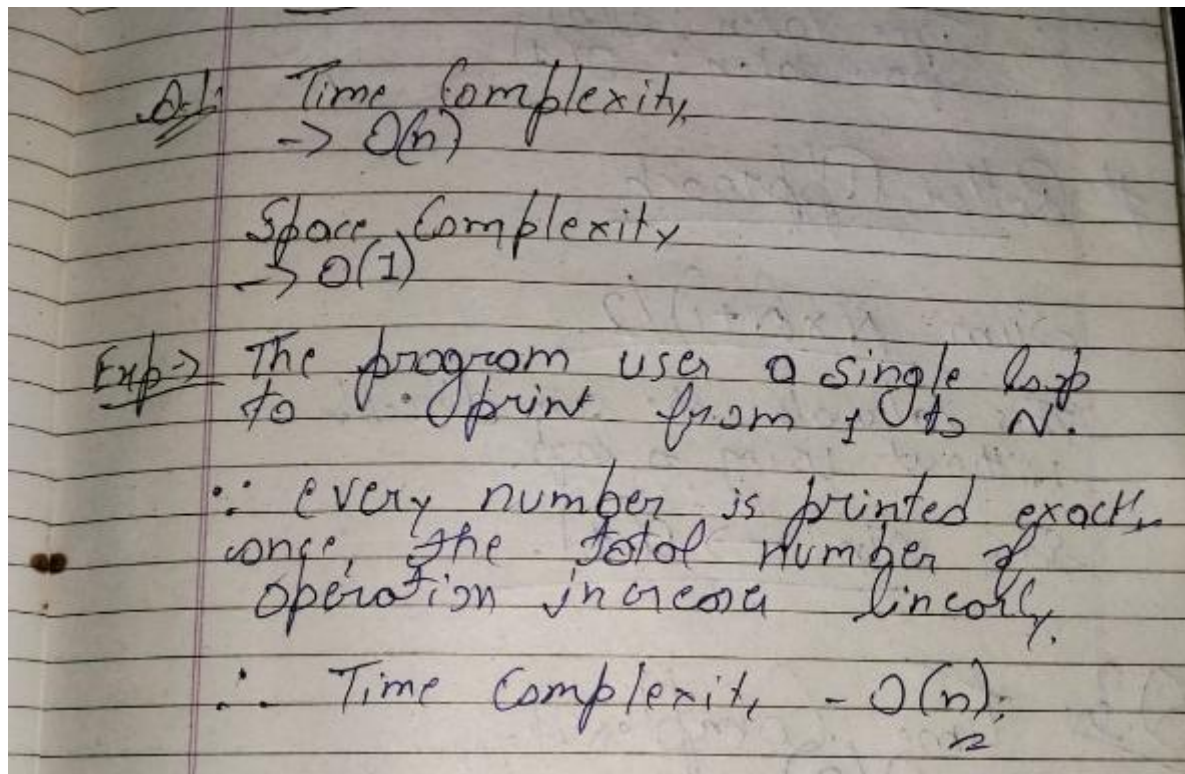
```
import java.util.Scanner;
public class PrintNumbers {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print(i + " ");
        }

        sc.close();
    }
}
```

OUTPUT

```
Command Prompt
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>java Nnumbers
Error: Could not find or load main class Nnumbers
Caused by: java.lang.ClassNotFoundException: Nnumbers

C:\Users\VAIBHAV SHARMA\Desktop\Week4module>java Nnumbers
Error: Could not find or load main class Nnumbers
Caused by: java.lang.ClassNotFoundException: Nnumbers

C:\Users\VAIBHAV SHARMA\Desktop\Week4module>javac PrintNumbers.java

C:\Users\VAIBHAV SHARMA\Desktop\Week4module>java PrintNumbers
3
1 2 3 4 5 6 7 8
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>
```

Question 2

```
import java.util.Scanner;

public class Sum {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int sum = 0;

        for (int i = 1; i <= n; i++) {
            sum += i;
        }

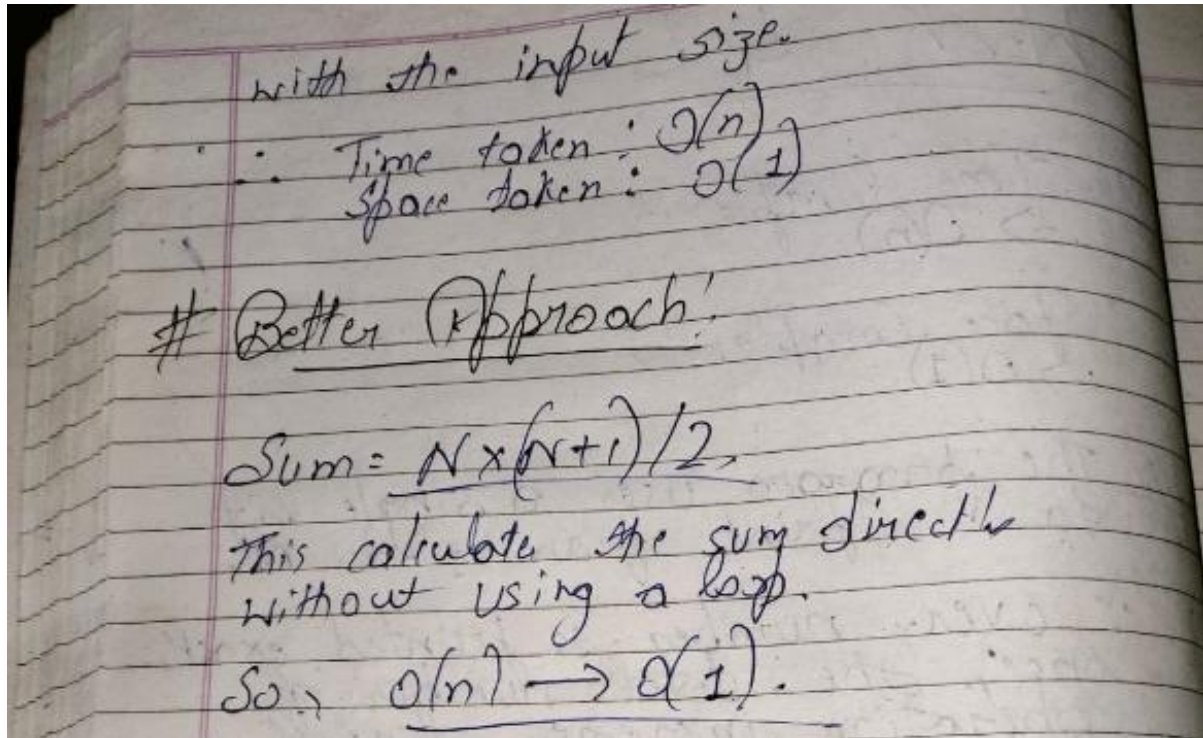
        System.out.println(sum);

        sc.close();
    }
}
```

Q2. Time Complexity,
→ $O(n)$.

Space Complexity,
→ $O(1)$

Exps) The program uses a loop to add all natural numbers from 1 to N . Since the loop executes once for each number, the total number of operations increases linearly.



OUTPUT:

```
Command Prompt
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>javac Sum.java
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>java Sum
10
55
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>
```

Question 3

```
import java.util.Scanner;

public class Exponent {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int a = sc.nextInt();
        int b = sc.nextInt();

        int result = 1;

        for (int i = 1; i <= b; i++) {
            result = result * a;
        }

        System.out.println(result);

        sc.close();
    }
}
```


Q.3 Time Complexity

→ $O(B)$.

Space Complexity,

→ $O(1)$

Exp → The program uses a loop that runs (B) times to calculate $A ^B $B </sup>$.$

∴ the time complexity is $O(B)$.

∴ few variables used so space is

Better Approach

→ Use Binary Exponentiation, which reduces the time complexity to $O(\log B)$.

OUTPUT:

```
Command Prompt
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>java Exponent
2
5
32
C:\Users\VAIBHAV SHARMA\Desktop\Week4module>
```